CHAPTER FIVE

Multiple Sequence Alignment

A natural extension of pairwise alignment is multiple sequence alignment, which is to align multiple related sequences to achieve optimal matching of the sequences. Related sequences are identified through the database similarity searching described in Chapter 4. As the process generates multiple matching sequence pairs, it is often necessary to convert the numerous pairwise alignments into a single alignment, which arranges sequences in such a way that evolutionarily equivalent positions across all sequences are matched.

There is a unique advantage of multiple sequence alignment because it reveals more biological information than many pairwise alignments can. For example, it allows the identification of conserved sequence patterns and motifs in the whole sequence family, which are not obvious to detect by comparing only two sequences. Many conserved and functionally critical amino acid residues can be identified in a protein multiple alignment. Multiple sequence alignment is also an essential prerequisite to carrying out phylogenetic analysis of sequence families and prediction of protein secondary and tertiary structures. Multiple sequence alignment also has applications in designing degenerate polymerase chain reaction (PCR) primers based on multiple related sequences.

It is theoretically possible to use dynamic programming to align any number of sequences as for pairwise alignment. However, the amount of computing time and memory it requires increases exponentially as the number of sequences increases. As a consequence, full dynamic programming cannot be applied for datasets of more than ten sequences. In practice, heuristic approaches are most often used. In this chapter, methodologies and applications of multiple sequence alignment are discussed.

SCORING FUNCTION

Multiple sequence alignment is to arrange sequences in such a way that a maximum number of residues from each sequence are matched up according to a particular scoring function. The scoring function for multiple sequence alignment is based on the concept of sum of pairs (SP). As the name suggests, it is the sum of the scores of all possible pairs of sequences in a multiple alignment based on a particular scoring matrix. In calculating the SP scores, each column is scored by summing the scores for all possible pairwise matches, mismatches and gap costs. The score of the entire

Downloaded from https://www.cambridge.org/core. University of Birmingham, on 18 Nov 2019 at 22:20:18, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/CBO9780511806087.006

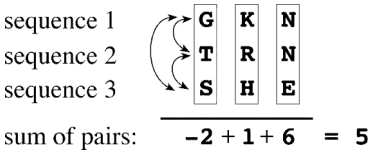

Figure 5.1: Given a multiple alignment of three sequences, the sum of scores is calculated as the sum of the similarity scores of every pair of sequences at each position. The scoring is based on the BLOSUM62 matrix (see Chapter 3). The total score for the alignment is 5, which means that the alignment is $2^5 = 32$ times more likely to occur among homologous sequences than by random chance.

alignment is the sum of all of the column scores (Fig. 5.1). The purpose of most multiple sequence alignment algorithms is to achieve maximum SP scores.

EXHAUSTIVE ALGORITHMS

As mentioned, there are exhaustive and heuristic approaches used in multiple sequence alignment. The exhaustive alignment method involves examining all possible aligned positions simultaneously. Similar to dynamic programming in pairwise alignment, which involves the use of a two-dimensional matrix to search for an optimal alignment, to use dynamic programming for multiple sequence alignment, extra dimensions are needed to take all possible ways of sequence matching into consideration. This means to establish a multidimensional search matrix. For instance, for three sequences, a three-dimensional matrix is required to account for all possible alignment scores. Back-tracking is applied through the three-dimensional matrix to find the highest scored path that represents the optimal alignment. For aligning N sequences, an N-dimensional matrix is needed to be filled with alignment scores. As the amount of computational time and memory space required increases exponentially with the number of sequences, it makes the method computationally prohibitive to use for a large data set. For this reason, full dynamic programming is limited to small datasets of less than ten short sequences. For the same reason, few multiple alignment programs employing this "brute force" approach are publicly available. A program called DCA, which uses some exhaustive components, is described below.

DCA (Divide-and-Conquer Alignment, http://bibiserv.techfak.uni-bielefeld.de/dca/) is a web-based program that is in fact semiexhaustive because certain steps of computation are reduced to heuristics. It works by breaking each of the sequences into two smaller sections. The breaking points are determined based on regional similarity of the sequences. If the sections are not short enough, further divisions are carried out. When the lengths of the sequences reach a predefined threshold, dynamic programming is applied for aligning each set of subsequences. The resulting short alignments are joined together head to tail to yield a multiple alignment of the entire length of all sequences. This algorithm provides an option of using a more heuristic procedure (fastDCA) to choose optimal cutting points so it can more rapidly handle a greater number of sequences. It performs global alignment and requires the input sequences to be of similar lengths and domain structures. Despite the use of heuristics, the program is still extremely computationally intensive and can handle only datasets of a very limited number of sequences.

Downloaded from https://www.cambridge.org/core. University of Birmingham, on 18 Nov 2019 at 22:20:18, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/CBO9780511806087.006

HEURISTIC ALGORITHMS

Because the use of dynamic programming is not feasible for routine multiple sequence alignment, faster and heuristic algorithms have been developed. The heuristic algorithms fall into three categories: progressive alignment type, iterative alignment type, and block-based alignment type. Each type of algorithm is described in turn.

Progressive Alignment Method

Progressive alignment depends on the stepwise assembly of multiple alignment and is heuristic in nature. It speeds up the alignment of multiple sequences through a multistep process. It first conducts pairwise alignments for each possible pair of sequences using the Needleman–Wunsch global alignment method and records these similarity scores from the pairwise comparisons. The scores can either be percent identity or similarity scores based on a particular substitution matrix. Both scores correlate with the evolutionary distances between sequences. The scores are then converted into evolutionary distances to generate a distance matrix for all the sequences involved. A simple phylogenetic analysis is then performed based on the distance matrix to group sequences based on pairwise distance scores. As a result, a phylogenetic tree is generated using the neighbor-joining method (see Chapter 11). The tree reflects evolutionary proximity among all the sequences.

It needs to be emphasized that the resulting tree is an approximate tree and does not have the rigor of a formally constructed phylogenetic tree (see Chapter 11). Nonetheless, the tree can be used as a guide for directing realignment of the sequences. For that reason, it is often referred to as a *guide tree*. According to the guide tree, the two most closely related sequences are first re-aligned using the Needleman–Wunsch algorithm. To align additional sequences, the two already aligned sequences are converted to a consensus sequence with gap positions fixed. The consensus is then treated as a single sequence in the subsequent step.

In the next step, the next closest sequence based on the guide tree is aligned with the consensus sequence using dynamic programming. More distant sequences or sequence profiles are subsequently added one at a time in accordance with their relative positions on the guide tree. After realignment with a new sequence using dynamic programming, a new consensus is derived, which is then used for the next round of alignment. The process is repeated until all the sequences are aligned (Fig. 5.2).

Probably the most well-known progressive alignment program is Clustal. Some of its important features are introduced next.

Clustal (www.ebi.ac.uk/clustalw/) is a progressive multiple alignment program available either as a stand-alone or on-line program. The stand-alone program, which runs on UNIX and Macintosh, has two variants, ClustalW and ClustalX. The W version provides a simple text-based interface and the X version provides a more user-friendly graphical interface.

Downloaded from https://www.cambridge.org/core. University of Birmingham, on 18 Nov 2019 at 22:20:18, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/CBO9780511806087.006

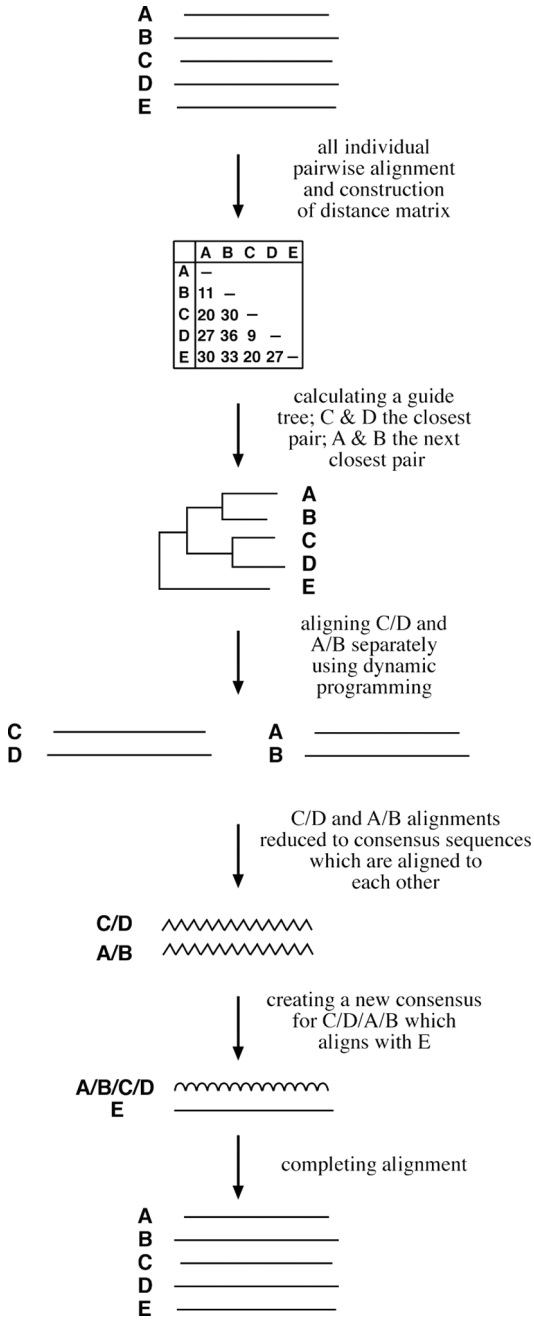

Figure 5.2: Schematic of a typical progressive alignment procedure (e.g., Clustal). Angled wavy lines represent consensus sequences for sequence pairs A/B and C/D. Curved wavy lines represent a consensus for A/B/C/D.

Downloaded from https://www.cambridge.org/core. University of Birmingham, on 18 Nov 2019 at 22:20:18, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/CBO9780511806087.006

One of the most important features of this program is the flexibility of using substitution matrices. Clustal does not rely on a single substitution matrix. Instead, it applies different scoring matrices when aligning sequences, depending on degrees of similarity. The choice of a matrix depends on the evolutionary distances measured from the guide tree. For example, for closely related sequences that are aligned in the initial steps, Clustal automatically uses the BLOSUM62 or PAM120 matrix. When more divergent sequences are aligned in later steps of the progressive alignment, the BLOSUM45 or PAM250 matrices may be used instead.

Another feature of Clustal is the use of adjustable gap penalties that allow more insertions and deletions in regions that are outside the conserved domains, but fewer in conserved regions. For example, a gap near a series of hydrophobic residues carries more penalties than the one next to a series of hydrophilic or glycine residues, which are common in loop regions. In addition, gaps that are too close to one another can be penalized more than gaps occurring in isolated loci.

The program also applies a weighting scheme to increase the reliability of aligning divergent sequences (sequences with less than 25% identity). This is done by down-weighting redundant and closely related groups of sequences in the alignment by a certain factor. This scheme is useful in preventing similar sequences from dominating the alignment. The weight factor for each sequence is determined by its branch length on the guide tree. The branch lengths are normalized by how many times sequences share a basal branch from the root of the tree. The obtained value for each sequence is subsequently used to multiply the raw alignment scores of residues from that sequence so to achieve the goal of decreasing the matching scores of frequent characters in a multiple alignment and thereby increasing the ones of infrequent characters.

Drawbacks and Solutions

The progressive alignment method is not suitable for comparing sequences of different lengths because it is a global alignment–based method. As a result of the use of affine gap penalties (see Chapter 3), long gaps are not allowed, and, in some cases, this may limit the accuracy of the method. The final alignment result is also influenced by the order of sequence addition. Another major limitation is the "greedy" nature of the algorithm: it depends on initial pairwise alignment. Once gaps introduced in the early steps of alignment, they are fixed. Any errors made in these steps cannot be corrected. This problem of "once an error, always an error" can propagate throughout the entire alignment. In other words, the final alignment could be far from optimal. The problem can be more glaring when dealing with divergent sequences. To alleviate some of the limitations, a new generation of algorithms have been developed, which specifically target some of the problems of the Clustal program.

T-Coffee (Tree-based Consistency Objective Function for alignment Evaluation; www.ch.embnet.org/software/TCoffee.html) performs progressive sequence alignments as in Clustal. The main difference is that, in processing a query, T-Coffee performs both global and local pairwise alignment for all possible pairs involved. The global pairwise alignment is performed using the Clustal program. The local pairwise

Downloaded from https://www.cambridge.org/core. University of Birmingham, on 18 Nov 2019 at 22:20:18, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/CBO9780511806087.006

alignment is generated by the Lalign program, from which the top ten scored alignments are selected. The collection of local and global sequence alignments are pooled to form a library. The consistency of the alignments is evaluated. For every pair of residues in a pair of sequences, a consistency score is calculated for both global and local alignments. Each pairwise alignment is further aligned with a possible third sequence. The result is used to refine the original pairwise alignment based on a consistency criterion in a process known as *library extension*. Based on the refined pairwise alignments, a distance matrix is built to derive a guide tree, which is then used to direct a full multiple alignment using the progressive approach.

Because an optimal initial alignment is chosen from many alternative alignments, T-Coffee avoids the problem of getting stuck in the suboptimal alignment regions, which minimizes errors in the early stages of alignment assembly. Benchmark assessment has shown that T-Coffee indeed outperforms Clustal when aligning moderately divergent sequences. However, it is also slower than Clustal because of the extra computing time necessary for the calculation of consistency scores. T-Coffee provides a graphical output of the alignment results, with colored boxes to display degree of agreement in the alignment library for various sequence regions.

DbClustal (http://igbmc.u-strasbg.fr:8080/DbClustal/dbclustal.html) is a Clustal-based database search algorithm for protein sequences that combines local and global alignment features. It first performs a BLASTP search for a query sequence. The resulting sequence alignment pairs above a certain threshold are analyzed to obtain *anchor points*, which are common conserved regions, by using a program called Ballast. A global alignment is subsequently generated by Clustal, which is weighted toward the anchor points. Since the anchor points are derived from local alignments, this strategy minimizes errors caused by the global alignment. The resulting multiple alignment is further evaluated by NorMD, which removes unrelated or badly aligned sequences from the multiple alignment. Thus, the final alignment should be more accurate than using Clustal alone. It also allows the incorporation of very long gaps for insertions and terminal extensions.

Poa (Partial order alignments, www.bioinformatics.ucla.edu/poa/) is a progressive alignment program that does not rely on guide trees. Instead, the multiple alignment is assembled by adding sequences in the order they are given. Instead of using regular sequence consensus, a partial order graph is used to represent a growing multiple alignment, in which identical residues in a column are condensed to a node resembling a knot on a rope and divergent residues are allowed to remain as such, allowing the rope to "bubble" (Fig. 5.3). The graph profile preserves all the information from the original alignment. Each time a new sequence is added, it is aligned with every sequence within the partial order graph individually using the Smith–Waterman algorithm. This allows the formation of a modified graph model, which is then used for the next cycle of pairwise alignment. By building such a graph profile, the algorithm maintains the information of the original sequences and eliminates the problem of error fixation as in the Clustal alignment. Poa is local alignment-based and has been shown to produce more accurate alignments than Clustal. Another advantage of this

Downloaded from https://www.cambridge.org/core. University of Birmingham, on 18 Nov 2019 at 22:20:18, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/CBO9780511806087.006

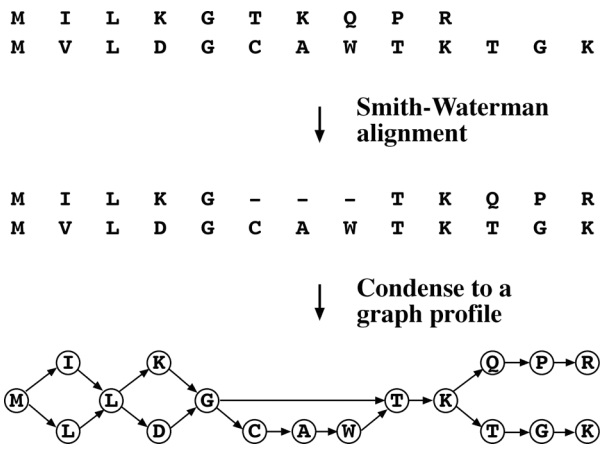

Figure 5.3: Conversion of a sequence alignment into a graphical profile in the Poa algorithm. Identical residues in the alignment are condensed as nodes in the partial order graph.

algorithm is its speed. It is reported to be able to align 5,000 sequences in 4 hours using a regular PC workstation. It is available both as an online program and as a stand-alone UNIX program.

PRALINE (http://ibivu.cs.vu.nl/programs/pralinewww/) is a web-based progressive alignment program. It first performs preprocessing of the input sequences by building profiles for each sequence. Profiles (see Chapter 6) can be interpreted as probability description of a multiple alignment. By default, the profiles are automatically generated using PSI-BLAST database searching (see Chapter 6). Each preprocessed profile is then used for multiple alignment using the progressive approach. However, this method does not use a guide tree in the successive enlargement of the alignment, but rather considers the closest neighbor to be joined to a larger alignment by comparing the profile scores. Because the profiles already incorporate information of distant relatives of each input sequence, this approach allows more accurate alignment of distantly related sequences in the original dataset. In addition, the program also has the feature to incorporate protein secondary structure information which is derived from state-of-the-art secondary structure prediction programs, such as PROF or SSPRO (see Chapter 14). The secondary structure information is used to modify the profile scores to help constrain sequence matching to the structured regions. PRALINE is perhaps the most sophisticated and accurate alignment program available. Because of the high complexity of the algorithm, its obvious drawback is the extremely slow computation.

Iterative Alignment

The iterative approach is based on the idea that an optimal solution can be found by repeatedly modifying existing suboptimal solutions. The procedure starts by producing a low-quality alignment and gradually improves it by iterative realignment through well-defined procedures until no more improvements in the alignment scores

Downloaded from https://www.cambridge.org/core. University of Birmingham, on 18 Nov 2019 at 22:20:18, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/CBO9780511806087.006

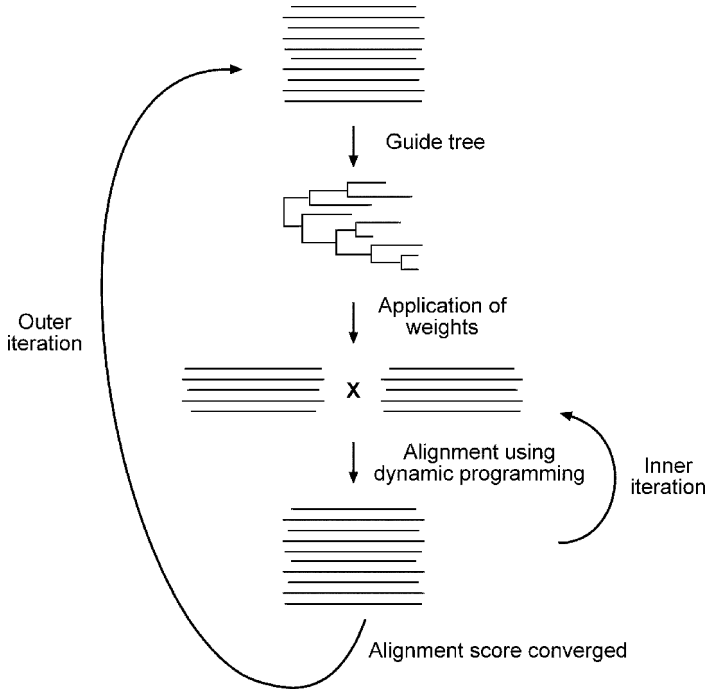

Figure 5.4: Schematic of iterative alignment procedure for PRRN, which involves two sets of iterations.

can be achieved. Because the order of the sequences used for alignment is different in each iteration, this method may alleviate the "greedy" problem of the progressive strategy. However, this method is also heuristic in nature and does not have guarantees for finding the optimal alignment. An example of iterative alignment is given below.

PRRN (http://prrn.ims.u-tokyo.ac.jp/) is a web-based program that uses a double-nested iterative strategy for multiple alignment. It performs multiple alignment through two sets of iterations: inner iteration and outer iteration. In the *outer iteration*, an initial random alignment is generated that is used to derive a UPGMA tree (see Chapter 11). Weights are subsequently applied to optimize the alignment. In the *inner iteration*, the sequences are randomly divided into two groups. Randomized alignment is used for each group in the initial cycle, after which the alignment positions in each group are fixed. The two groups, each treated as a single sequence, are then aligned to each other using global dynamic programming. The process is repeated through many cycles until the total SP score no longer increases. At this point, the resulting alignment is used to construct a new UPGMA tree. New weights are applied to optimize alignment scores. The newly optimized alignment is subject to further realignment in the inner iteration. This process is repeated over many cycles until there is no further improvement in the overall alignment scores (Fig. 5.4).

Downloaded from https://www.cambridge.org/core. University of Birmingham, on 18 Nov 2019 at 22:20:18, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/CBO9780511806087.006

Block-Based Alignment

The progressive and iterative alignment strategies are largely global alignment based and may therefore fail to recognize conserved domains and motifs (see Chapter 7) among highly divergent sequences of varying lengths. For such divergent sequences that share only regional similarities, a local alignment based approach has to be used. The strategy identifies a block of ungapped alignment shared by all the sequences, hence, the block-based local alignment strategy. Two block-based alignment programs are introduced below.

DIALIGN2 (http://bioweb.pasteur.fr/seqanal/interfaces/dialign2.html) is a web-based program designed to detect local similarities. It does not apply gap penalties and thus is not sensitive to long gaps. The method breaks each of the sequences down to smaller segments and performs all possible pairwise alignments between the segments. High-scoring segments, called *blocks*, among different sequences are then compiled in a progressive manner to assemble a full multiple alignment. It places emphasis on block-to-block comparison rather than residue-to-residue comparison. The sequence regions between the blocks are left unaligned. The program has been shown to be especially suitable for aligning divergent sequences with only local similarity.

Match-Box (www.sciences.fundp.ac.be/biologie/bms/matchbox_submit.shtml) is a web-based server that also aims to identify conserved blocks (or boxes) among sequences. The program compares segments of every nine residues of all possible pairwise alignments. If the similarity of particular segments is above a certain threshold across all sequences, they are used as an anchor to assemble multiple alignments; residues between blocks are unaligned. The server requires the user to submit a set of sequences in the FASTA format and the results are returned by e-mail.

PRACTICAL ISSUES

Protein-Coding DNA Sequences

As mentioned in the Chapter 4, alignment at the protein level is more sensitive than at the DNA level. Sequence alignment directly at the DNA level can often result in frameshift errors because in DNA alignment gaps are introduced irrespective of codon boundaries. Therefore, in the process of achieving maximum sequence similarity at the DNA level, mismatches of genetic codons occur that violate the accepted evolutionary scenario that insertions or deletions occur in units of codons. The resulting alignment can thus be biologically unrealistic. The example in Figure 5.5 shows how such errors can occur when two sequences are being compared at the protein and DNA levels.

For that reason, sequence alignment at the protein level is much more informative for functional and evolutionary analysis. However, there are occasions when sequence

Downloaded from https://www.cambridge.org/core. University of Birmingham, on 18 Nov 2019 at 22:20:18, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/CBO9780511806087.006

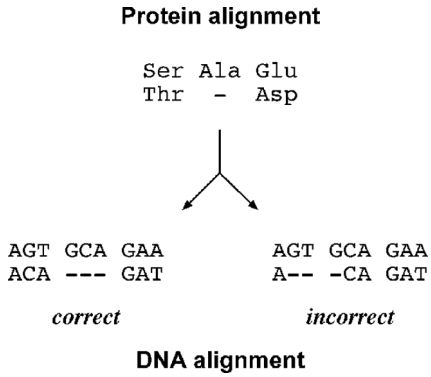

Figure 5.5: Comparison of alignment at the protein level and DNA level. The DNA alignment on the left is the correct one and consistent with amino acid sequence alignment, whereas the DNA alignment on the right, albeit more optimal in matching similar residues, is incorrect because it disregards the codon boundaries.

alignment at the DNA level is often necessary, for example, in designing PCR primers and in constructing DNA-based molecular phylogenetic trees.

Because conducting alignment directly at the DNA level often leads to errors, DNA can be translated into an amino acid sequence before carrying out alignment to avoid the errors of inserting gaps within codon boundaries. After alignment of the protein sequences, the alignment can be converted back to DNA alignment while ensuring that codons of the DNA sequences line up based on corresponding amino acids. The following are two web-based programs that allow easy conversion from protein alignment to DNA alignment.

RevTrans (www.cbs.dtu.dk/services/RevTrans/) takes a set of DNA sequences, translates them, aligns the resulting protein sequences, and uses the protein alignment as a scaffold for constructing the corresponding DNA multiple alignment. It also allows the user to provide multiple protein alignment for greater control of the alignment process. PROTA2DNA (http://bioweb.pasteur.fr/seqanal/interfaces/protal2dna.html) aligns DNA sequences corresponding to a protein multiple alignment.

Editing

No matter how good an alignment program seems, the automated alignment often contains misaligned regions. It is imperative that the user check the alignment carefully for biological relevance and edit the alignment if necessary. This involves introducing or removing gaps to maximize biologically meaningful matches. Sometimes, portions that are ambiguously aligned and deemed to be incorrect have to deleted. In manual editing, empirical evidence or mere experience is needed to make corrections on an alignment. One can simply use a word processor to edit the text-based alignment. There are also dedicated software programs that assist in the process.

BioEdit (www.mbio.ncsu.edu/BioEdit/bioedit.html) is a multifunctional sequence alignment editor for Windows. It has a coloring scheme for nucleotide or amino acid residues that facilitates manual editing. In addition, it is able to do BLAST searches, plasmid drawing, and restriction mapping.

Downloaded from https://www.cambridge.org/core. University of Birmingham, on 18 Nov 2019 at 22:20:18, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/CBO9780511806087.006

Rascal (http://igbmc.u-strasbg.fr/PipeAlign/Rascal/rascal.html) is a web-based program that automatically refines a multiple sequence alignment. It is part of the PipeAlign package. It is able to identify misaligned regions and realign them to improve the quality of the alignment. It works by dividing the input alignment into several regions horizontally and vertically to identify well-aligned and poorly aligned regions using an internal scoring scheme. Regions below certain thresholds are considered misaligned and are subsequently realigned using the progressive approach. The overall quality of the alignment is then reassessed. If necessary, certain regions are further realigned. The program also works in conjunction with NorMD, which validates the refined alignment and identifies potentially unrelated sequences for removal.

Format Conversion

In many bioinformatics analyses, in particular, phylogenetic analysis, it is often necessary to convert various formats of sequence alignments to the one acceptable by an application program. The task of format conversion requires a program to be able to read a multiple alignment in one format and rewrite it into another while maintaining the original alignment. This is a different task from simply converting the format of individual unaligned sequences. The BioEdit program mentioned is able to save an alignment in a variety of different formats. In addition, the Readseq program mentioned in Chapter 2 is able to perform format conversion of multiple alignment.

Readseq (http://iubio.bio.indiana.edu/cgi-bin/readseq.cgi/) is a web-based program that is able to do both simple sequence format conversion as well as alignment format conversions. The program can handle formats such as MSF, Phylip, Clustal, PAUP, and Pretty.

SUMMARY

Multiple sequence alignment is an essential technique in many bioinformatics applications. Many algorithms have been developed to achieve optimal alignment. Some programs are exhaustive in nature; some are heuristic. Because exhaustive programs are not feasible in most cases, heuristic programs are commonly used. These include progressive, iterative, and block-based approaches. The progressive method is a stepwise assembly of multiple alignment according to pairwise similarity. A prominent example is Clustal, which is characterized by adjustable scoring matrices and gap penalties as well as by the application of weighting schemes. The major shortcoming of the program is its "greediness," which relates to error fixation in the early steps of computation. To remedy the problem, T-Coffee and DbClustal have been developed that combine both global and local alignment to generate more sensitive alignment. Another improvement on the traditional progressive approach is to use graphic profiles, as in Poa, which eliminate the problem of error fixation. Praline is profile based and has the capacity to restrict alignment based on protein structure information and is thus much more accurate than Clustal. The iterative approach works by repetitive

Downloaded from https://www.cambridge.org/core. University of Birmingham, on 18 Nov 2019 at 22:20:18, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/CBO9780511806087.006

refinement of suboptimal alignments. The block-based method focuses on identifying regional similarities. It is important to keep in mind that no alignment program is absolutely guaranteed to find correct alignment, especially when the number of sequences is large and the divergence level is high. The alignment resulting from automated alignment programs often contains errors. The best approach is to perform alignment using a combination of multiple alignment programs. The alignment result can be further refined manually or using Rascal. Protein-encoding DNA sequences should preferably be aligned at the protein level first, after which the alignment can be converted back to DNA alignment.

FURTHER READING

Apostolico, A., and Giancarlo, R. 1998. Sequence alignment in molecular biology. *J. Comput. Biol*. 5:173–96.

Gaskell, G. J. 2000. Multiple sequence alignment tools on the Web. *Biotechniques* 29:60–2.

Gotoh, O. 1999. Multiple sequence alignment: Algorithms and applications. *Adv. Biophys*. 36:159–206.

Lecompte, O., Thompson, J. D., Plewniak, F., Thierry, J., and Poch, O. 2001. Multiple alignment of complete sequences (MACS) in the post-genomic era. *Gene* 270:17–30.

Morgenstern, B. 1999. DIALIGN 2: improvement of the segment-to-segment approach to multiple sequence alignment. *Bioinformatics* 15:211–8.

Morgenstern, B., Dress, A., and Werner T. 1996. Multiple DNA and protein sequence alignment based on segment-to-segment comparison. *Proc. Natl. Acad. Sci. U S A* 93:12098–103.

Mullan, L. J. 2002. Multiple sequence alignment – The gateway to further analysis. *Brief. Bioinform*. 3:303–5.

Nicholas, H. B. Jr., Ropelewski, A. J., and Deerfield, D. W. II. 2002. Strategies for multiple sequence alignment. *Biotechniques* 32:572–91.

Notredame, C. 2002. Recent progress in multiple sequence alignment: A survey. *Pharmacogenomics* 3:131–44.

Notredame, C., Higgins, D. G., and Heringa, J. 2000. T-Coffee: A novel method for fast and accurate multiple sequence alignment. *J. Mol. Biol.* 302:205–17.

Thompson, J. D., Higgins, D. G., and Gibson, T. J. 1994. CLUSTAL W: Improving the sensitivity of progressive multiple sequence alignment through sequence weighting, position-specific gap penalties and weight matrix choice. *Nucleic Acids Res*. 22:4673–80.

Thompson, J. D., Plewniak, F., and Poch, O. 1999. A comprehensive comparison of multiple sequence alignment programs. *Nucleic Acids Res*. 27:2682–90.

Downloaded from https://www.cambridge.org/core. University of Birmingham, on 18 Nov 2019 at 22:20:18, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/CBO9780511806087.006